

Database Management Systems

Shamim Ahmad



Referential Integrity

Referential Integrity

- Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
 - ★ Example: If “Rajshahi” is a branch name appearing in one of the tuples in the *account* relation, then there exists a tuple in the *branch* relation for branch “Rajshahi”.

Checking Referential Integrity on Database Modification

- The following tests must be made in order to preserve the following **referential integrity constraint**:

<i>account-number</i>	<i>branch-name</i>	<i>balance</i>
A-101	Downtown	500
A-102	Perryridge	400
A-201	Brighton	900
A-215	Mianus	700
A-217	Brighton	750
A-222	Redwood	700
A-305	Round Hill	350

<i>branch_name</i>	<i>branch_city</i>	<i>assets</i>
Brighton	Brooklyn	7100000
Downtown	Brooklyn	9000000
Mianus	Horseneck	400000
North Town	Rye	3700000
Perryridge	Horseneck	1700000
Pownal	Bennington	300000
Redwood	Palo Alto	2100000
Round Hill	Horseneck	8000000

Referential Integrity in SQL – Example (Cont.)

create table *account*

(account-number char(10),

branch-name char(15),

balance integer,

primary key (*account-number*),

foreign key (*branch-name*) **references** *branch*)

create table *depositor*

(customer-name char(20),

account-number char(10),

primary key (*customer-name*, *account-number*),

foreign key (*account-number*) **references** *account*,

foreign key (*customer-name*) **references** *customer*)

Triggers

Triggers

- A **trigger** is a statement that is executed **automatically** by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
 - ★ **Specify the conditions** under which the trigger is to be executed.
 - ★ **Specify the actions** to be taken when the trigger executes.
-

Authorization

Forms of authorization on parts of the database:

- **Read authorization** - allows reading, but not modification of data.
- **Insert authorization** - allows insertion of new data, but not modification of existing data.
- **Update authorization** - allows modification, but not deletion of data.
- **Delete authorization** - allows deletion of data

Views

◆ Definition

A View is a stored query that acts as a window to data in other tables. It behaves like a table in queries — you can **SELECT**, **JOIN**, or even **UPDATE** (in some cases) through it.

View Example

★ The *cust-loan* view is defined in SQL as follows:

```
create view cust-loan as  
  select branchname, customer-name  
  from   borrower, loan  
  where borrower.loan-number = loan.loan-number
```

sql

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Key Features

- 1.Virtual Table** – A view does not store data physically; it retrieves data from base tables each time it's accessed.
- 2.Simplified Querying** – Complex SQL queries can be saved as a view for easier reuse.
- 3.Security** – It can restrict user access to specific columns or rows (e.g., hiding salary information).
- 4.Consistency** – If base tables change, the view automatically reflects the updated data.
- 5.Updatability** – Some views can be updated if based on a single table and no aggregate or group operations are used.

EmpID	Name	Department	Salary
1	Rahim	IT	60000
2	Karim	HR	50000
3	Sultana	IT	70000

EmpID	Name	Department	Salary
1	Rahim	IT	60000
2	Karim	HR	50000
3	Sultana	IT	70000

```
CREATE VIEW IT_Employees AS  
SELECT Name, Salary  
FROM Employees  
WHERE Department = 'IT';
```

Now you can use:

sql

```
SELECT * FROM IT_Employees;
```

Authorization and Views

- Users can be given authorization on views, without being given any authorization on the relations used in the view definition
- Ability of views to **hide data serves** both to simplify usage of the system and to enhance security by allowing users access only to data they need for their job
- A combination of relational-level security and view-level security can be used to limit a user's access to precisely the data that user needs.

Aspect	Normal View	Indexed View
Definition	A virtual table created from a SQL query; stores no data physically.	A view that has a clustered index created on it; the result is materialized (physically stored) .
Storage	No physical storage; result is computed on demand.	Requires physical storage for indexed data.
Performance	Querying may be slower for large tables (because view is calculated each time).	Query performance is much faster for aggregate/complex queries due to stored index.
Use Case	Good for simple queries, security, reusability.	Good for performance optimization on large data and aggregate computations.
Data Update	Always shows the latest data automatically.	Automatically updated but has overhead during INSERT/UPDATE/DELETE operations.
Indexing	Cannot have an index directly.	Requires at least one unique clustered index .
Materialization	Not materialized (virtual).	Materialized (stored physically).
Maintenance Cost	Low maintenance cost.	Higher maintenance due to index updates.
Restrictions	Very few restrictions.	Many restrictions—e.g., cannot use COUNT(*), TOP, DISTINCT, outer joins, subqueries, etc.